

Concurrent Web Server: Thread Pool Architecture with Size-Based Scheduling

Ishit Arhatia

MS Data Science

Khoury College of Computer Sciences
Northeastern University

Helena Zhuo

MS Computer Science

Khoury College of Computer Sciences
Northeastern University

Rudrajit Banerjee

MS Computer Science

Khoury College of Computer Sciences
Northeastern University

Tomas Colorado

MS Computer Science

Khoury College of Computer Sciences
Northeastern University

Abstract—This paper presents a performance evaluation of size-based request scheduling in a multi-threaded web server with producer-consumer architecture, built around a fixed-size thread pool and a bounded buffer, following the architecture described in the OSTEP curriculum [1]. The main thread continuously accepts incoming HTTP connections and enqueues requests into a shared buffer; a pool of worker threads dequeue and service those requests according to a configurable scheduling policy. Our research extension investigates *size-based scheduling*—specifically comparing First-Come-First-Served (FCFS) against Shortest-File-First (SFF)—to evaluate their impact on mean response time, tail latency, and system behavior under varying load conditions. Drawing on established research demonstrating that web file sizes follow heavy-tailed distributions where most requests are small but most bytes originate from a small number of large files [3], we hypothesize that SFF scheduling will significantly reduce mean response time without substantially penalizing large-file requests. We validate this hypothesis through controlled experiments measuring throughput, response time distribution (mean, p50, p95, p99), and system behavior across different levels of concurrent connections and file-size mixes.

Index Terms—Concurrent web server, thread pool, bounded buffer, producer-consumer, scheduling policy, First-Come-First-Served (FCFS), Shortest-File-First (SFF), size-based scheduling, heavy-tailed workloads, POSIX threads.

I. INTRODUCTION

Modern web servers must handle thousands of simultaneous client connections while maintaining low latency and high throughput. A foundational design choice in any such system is how to move work from the single point of connection acceptance—a listening socket—to a pool of threads that can service requests in parallel. The canonical solution, described by Welsh et al. in the SEDA architecture [2] and formalized in the OSTEP curriculum, is the *producer-consumer bounded buffer*: the main thread produces connection descriptors into a fixed-capacity queue, and a fixed-size pool of worker threads consume from it. This decoupling keeps the acceptance loop fast and provides natural backpressure when workers are saturated.

Once this architecture is in place, a second design choice emerges: in what *order* should workers dequeue requests? The simplest policy is FCFS (First-Come-First-Served), which processes requests strictly in arrival order. FCFS is fair, simple to implement, and free of starvation. However, web workloads

are not uniform. Crovella and Bestavros showed that World Wide Web traffic is self-similar and that file sizes follow heavy-tailed (Pareto-like) distributions [5]: the vast majority of objects are small (a few kilobytes), yet a small number of large files account for most bytes transferred. Under these conditions, FCFS performs poorly—a single large file can block dozens of waiting small requests behind it.

Size-based scheduling policies such as Shortest-File-First (SFF) exploit this skew. By allowing workers to select the smallest pending request from the buffer rather than the oldest, small requests jump ahead of large ones, completing quickly and clearing queue capacity. Harchol-Balter et al. proved theoretically and through trace-driven simulation that this approach can reduce mean response time by orders of magnitude under moderate-to-high load without materially starving large requests [3]. Schroeder and Harchol-Balter further showed that the performance gap widens dramatically as server load approaches capacity [4]—exactly the operating regime that matters most.

This paper asks: *does SFF scheduling meaningfully outperform FCFS in a POSIX thread pool web server serving a realistic heavy-tailed workload, and what are the fairness trade-offs for large-file clients?* We answer this question by building a fully conformant G.5 OSTEP web server and subjecting it to automated load tests using an external benchmarking rig.

The remainder of this paper is organized as follows. Section II surveys the relevant literature grouped by theme. Section III describes our system architecture and key design decisions. Section IV defines our experimental methodology and benchmarking rig. Section V presents baseline results and preliminary findings. Section VI summarizes our work and outlines next steps.

II. RELATED WORK

Related work naturally divides into three themes: the architecture of concurrent servers, the theory of size-based scheduling, and modern production-scale scheduling systems that validate or extend these ideas.

A. Architecture of Concurrent Servers

The foundation of our design lies in the threading model introduced by Anderson et al. [6], which identified the core tension in concurrent systems: kernel-level threads are expensive (context switches on the order of $100\mu\text{s}$) but integrate correctly with blocking I/O, while user-level threads are an order of magnitude faster but stall the entire process on any blocking system call. Our implementation resolves this tension using POSIX `pthread`s with a fixed-size pool, accepting kernel-thread cost in exchange for correct semantics when workers block on file I/O.

Building on this threading foundation, Welsh, Culler, and Brewer introduced SEDA [2]—a Staged Event-Driven Architecture that partitions server logic into a pipeline of stages, each with its own thread pool and queue. SEDA’s Haboob HTTP server implementation used a form of shortest connection first scheduling within each stage, directly motivating our SFF policy. The producer-consumer bounded buffer at the heart of our design is a single-stage simplification of SEDA’s multi-stage pipeline.

von Behren et al. later challenged SEDA’s event-driven philosophy with Capriccio [7], arguing that user-level thread systems can match event-driven performance while retaining the simpler sequential programming model. Capriccio introduced resource-aware scheduling, where the scheduler used compiler-annotated resource usage predictions to make smarter thread selection decisions—conceptually analogous to our use of `stat()` to obtain file size before enqueueing a request.

B. Size-Based Scheduling Theory

The theoretical backbone of our research extension is the work of Harchol-Balter et al. [3], who formalized size-based scheduling for web servers as an application of Shortest-Remaining-Processing-Time (SRPT) queuing theory. The key insight is that, under heavy-tailed workloads, prioritizing short jobs (small files) dramatically reduces mean sojourn time because the population of competing small jobs is large. The paper’s trace-driven simulations on real web server logs showed mean response time improvements of $2\text{--}7\times$ over FCFS at moderate-to-high loads.

Schroeder and Harchol-Balter [4] extended this analysis to overload conditions, proving that FCFS response times explode exponentially as load approaches server capacity, while SFF-style policies remain stable. This is critical for our experimental design: we must measure not only average conditions but also behavior as concurrent connections increase toward saturation. The paper also characterized the “starvation myth”—large requests are rarely penalized significantly under SFF because they are statistically infrequent in heavy-tailed workloads.

The heavy-tailed nature of web traffic itself was characterized by Crovella and Bestavros [5], who demonstrated through empirical analysis of HTTP traces that file size distributions are Pareto-like with shape parameter $\alpha \approx 1.1$. This self-similarity is the fundamental property that makes size-based scheduling effective and is the reason our benchmarking rig

generates workloads with mixed small and large files rather than uniform sizes.

C. Modern Production Scheduling Systems

Recent systems work validates and extends the scheduling ideas above at production scale. Ousterhout et al.’s Shenango [11] achieves microsecond-scale thread scheduling via a dedicated IOKernel that reallocates CPU cores across applications as request rates change. While Shenango operates at a finer granularity than our thread pool model, its architecture directly parallels ours: the IOKernel is analogous to our main acceptor thread, and the per-application runtime pools are analogous to our worker thread pool. Shenango’s evaluation confirmed that scheduling policy—not mechanism overhead—is the primary determinant of tail latency.

Humphries et al.’s ghOst [9], deployed in production at Google, decouples scheduling policy from the kernel enforcement mechanism entirely, allowing user-space agents to implement FIFO, priority, or preemptive policies by submitting transactions to the kernel. This validates our design choice of making scheduling configurable via a runtime flag (`-s schedalg`) rather than hardcoding one policy. Demoulin et al.’s Perséphone [10] demonstrated that size-based scheduling at the microsecond scale—for workloads with heavy-tailed service time distributions similar to our file size distribution—continues to provide substantial tail latency improvements over FIFO, directly corroborating the theoretical predictions of Harchol-Balter.

Lampson’s foundational design heuristics [8] also informed our architectural choices. The principle of splitting resources in a fixed way justifies our fixed-size thread pool over dynamic thread creation, and the advice to handle the normal and worst case separately underpins our SFF rationale: the normal case (small files) should complete immediately, while the exceptional case (large files) must not block them.

III. SYSTEM DESIGN

A. Architectural Overview

Our system implements the G.5 OSTEP concurrent web server specification. The server accepts connections on a TCP socket and distributes HTTP requests to a fixed pool of worker threads via a shared bounded buffer, following the producer-consumer pattern. Three configurable parameters govern runtime behavior: `-t N` sets the thread pool size (default: 1), `-b N` sets the buffer capacity (default: 1), and `-s {FCFS|SFF}` selects the scheduling policy.

B. Thread Pool and Bounded Buffer

The bounded buffer is implemented as a fixed-capacity circular array protected by a single `pthread_mutex_t` and two condition variables: `notFull` (signaled by consumers after dequeue) and `notEmpty` (signaled by the producer after enqueue). A fixed-size array is used rather than a dynamic queue to ensure memory-safe operation under concurrent access without requiring heap allocation inside the critical section.

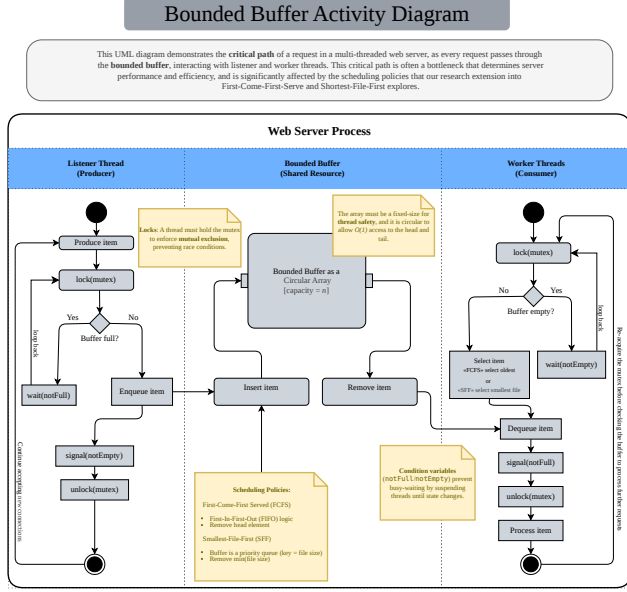


Fig. 1. Activity diagram of the bounded buffer. The listener thread (producer) enqueues connection descriptors; worker threads (consumers) dequeue according to the active scheduling policy. Mutex and condition variables (notFull/notEmpty) prevent busy-waiting and enforce mutual exclusion.

Each slot in the buffer holds a `request_t` struct that carries all the information a worker thread needs to service a request without accessing any shared state after dequeue. The `client_fd` field is the accepted socket descriptor passed from the main thread; `filename` holds the sanitized URI path extracted during initial parsing; `file_size` stores the result of the `stat()` call made by the producer before acquiring the mutex; and `arrival_ms` records the enqueue timestamp used by FCFS to preserve arrival order. The struct is defined as follows:

```
1 typedef struct {
2     int    client_fd;    /* accepted socket
3                          descriptor */
4     char   filename[256]; /* parsed URI path
5                          */
6     off_t  file_size;    /* from stat(), set on
7                          enqueue */
8     long   arrival_ms;   /* timestamp for FCFS
9                          order */
10 } request_t;
```

The `file_size` field is populated by the producer via a `stat()` call *before* entering the critical section, so that no filesystem operation occurs while the mutex is held.

C. Scheduling Policies

FCFS. The consumer removes the element at the buffer's logical head—a standard circular-queue dequeue in $O(1)$.

SFF. The consumer scans all `count` slots in the buffer, identifies the entry with the minimum `file_size`, removes

HTTP Request-Response Sequence Diagram

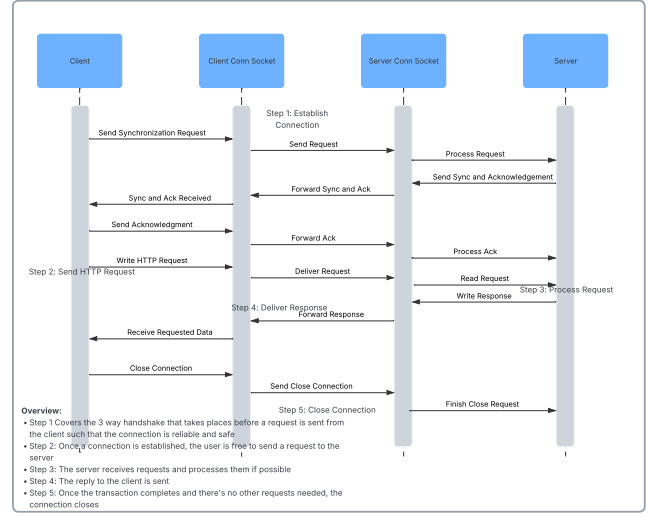


Fig. 2. Sequence diagram of the full HTTP request-response lifecycle, including the TCP three-way handshake (Step 1), request transmission (Step 2), server processing (Step 3), response delivery (Step 4), and connection teardown (Step 5).

it, and shifts the remaining elements to close the gap. This is an $O(n)$ scan where n is the current buffer occupancy. Given typical buffer sizes (8–64 slots), the overhead is negligible compared to the disk I/O that follows.

The scheduling selection is encapsulated in a single function pointer assigned at startup from the `-s` argument, so the rest of the worker code is policy-agnostic.

D. HTTP Request-Response Lifecycle

Figure 2 details the five phases of each request. On connection establishment (Step 1), the main thread completes the TCP three-way handshake and obtains a connected socket descriptor. This descriptor is enqueued into the bounded buffer (Step 2). A worker thread dequeues it (Step 3), reads and parses the HTTP request line, performs a `stat()` to verify the file exists and obtain its size, then maps and transmits the file contents with an appropriate status line (Step 4). The connection is closed after the response unless `Connection: keep-alive` is negotiated (Step 5). The server returns 200 OK for valid requests and 404 Not Found for missing resources.

E. Request-to-Job Dispatch

Figure 3 illustrates the system from a kernel-interaction perspective. The Dispatcher component in our implementation is the bounded buffer's dequeue logic. This is where our research intervention occurs: by changing the selection criterion at Step 4 from arrival-order (FCFS) to minimum-file-size (SFF), we hypothesize a significant reduction in mean response time for small-file clients under load.

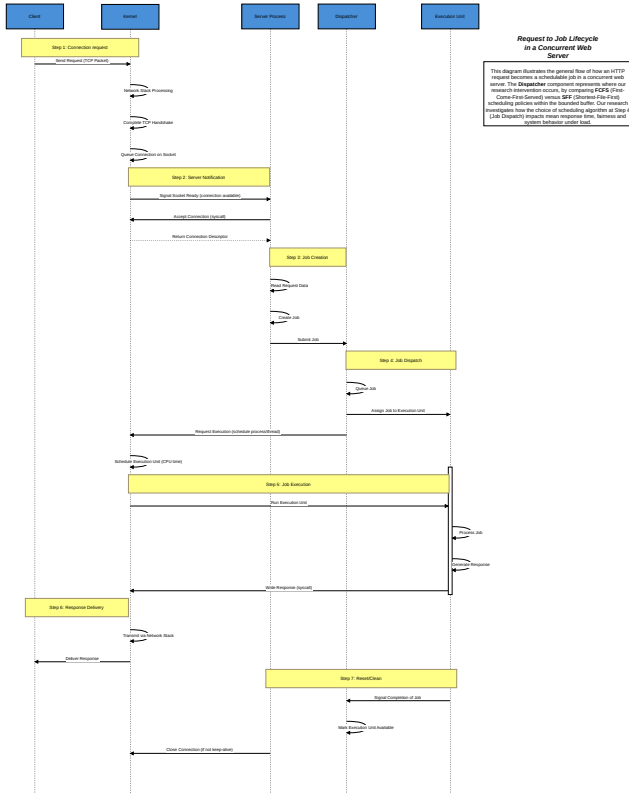


Fig. 3. The request-to-job lifecycle across the Client, Kernel, Server Process, Dispatcher, and Execution Unit. Our research intervention occurs at Step 4 (Job Dispatch), where the Dispatcher applies either FCFS or SFF to select the next request from the bounded buffer.

F. Source Organization

The codebase is organized into six focused modules, each with a single responsibility, plus a top-level `Makefile`. This separation ensures that the bounded buffer, thread pool, and HTTP logic remain independently testable and that scheduling policy changes are confined to `src/buffer.c` without touching any other file. Table I lists each module and its role.

TABLE I
SOURCE FILE RESPONSIBILITIES

File	Responsibility
<code>main.c</code>	Arg parsing, socket setup, accept loop
<code>bounded_buffer.c/h</code>	Buffer, mutex, condvars, FCFS/SFF
<code>thread_pool.c/h</code>	Pool init, thread creation, teardown
<code>http.c/h</code>	HTTP parsing, file send, status codes
<code>mock_client.c</code>	Correctness testing client
<code>Makefile</code>	<code>make / make clean</code>

G. Concurrency Correctness

All accesses to the shared bounded buffer pass through the mutex. Condition variables are checked inside `while` loops

(not `if` statements) to guard against spurious wakeups. The producer holds the mutex only while inserting into the array; the `stat()` call is performed beforehand. Workers hold the mutex only while scanning/removing from the buffer; all file I/O occurs after releasing the lock. This minimizes critical section duration and avoids holding the mutex across blocking calls.

IV. METHODOLOGY

A. Benchmarking Rig

All performance measurements are collected externally, treating the server as a black box. We use `wrk` [12] as our primary load generator because it supports configurable concurrency, duration-based runs, and Lua scripting for mixed-file-size workloads. Scripts are located in `/benchmarks/` and are fully automated: a single invocation of `benchmark.sh` creates the small and large test files using the `dd` Unix utility, starts the server, runs all experiments, tears down the server, and writes raw output to `/benchmarks/raw/`.

B. Experimental Variables

Independent variables:

- **Scheduling policy:** FCFS vs. SFF (primary comparison).
- **Concurrency level:** number of simultaneous `wrk` connections: $\{1, 10, 25, 50, 100, 200\}$.
- **File size mix:** uniform small (4 KB), uniform large (1 MB), and heavy-tailed (90% small / 10% large by request count).

Dependent variables:

- Throughput (requests per second).
- Response time: mean, p50, p95, p99 (milliseconds).
- Error rate (connection resets, timeouts).

Controlled variables: thread pool size fixed at 4; buffer capacity fixed at 16; same VM and OS image; no background processes during measurement; server and client run on the same host to eliminate network variability.

C. Experimental Protocol

Each configuration is run for 30 seconds. Before each individual trial's measurement window, a 5-second warm-up period is performed against the already-running server (since the server is restarted between trials, each trial starts cold and requires its own warm-up). We collect five independent trials per configuration, reporting the mean and standard deviation. Between trials the server is restarted to clear any warm state. Configurations where the server reaches connection saturation — indicated by a surge in connection refusal errors reported by `wrk` — are noted as such in the results rather than omitted, as saturation behavior itself is a meaningful data point for comparing FCFS and SFF under overload.

D. Environment

V. PRELIMINARY RESULTS

A. Baseline System Verification

The base system (FCFS only) was verified to correctly serve static files to a browser, handle concurrent connections without

TABLE II
MEASUREMENT ENVIRONMENT

Component	Specification
Host machine	Windows 10
Virtualization	Oracle VirtualBox
Guest OS	Ubuntu 22.04 LTS (Linux 5.15)
Guest resources	4 vCPUs, 8 GB RAM
Compiler	GCC 11.4, -O2
Load generator	wrk 4.1.03

data corruption or deadlock, and return appropriate HTTP status codes (200, 404).

B. Baseline Performance Numbers

Table III shows baseline FCFS measurements under the three file-size workloads at selected concurrency levels.

TABLE III
FCFS BASELINE: MEAN RESPONSE TIME (MS) AND THROUGHPUT (REQ/S)

Workload	Conc.	Mean RT (ms)	p99 (ms)	Req/s
Small (4 KB)	10	0.56	3.15	15,122
Small (4 KB)	100	3.55	6.81	26,715
Large (1 MB)	10	6.64	17.97	1,462
Large (1 MB)	100	56.34	87.61	1,765
Heavy-tailed	10	1.21	5.97	8,074
Heavy-tailed	100	8.79	16.23	11,149

Values represent the mean across five independent trials. Raw wrk output is committed to `/benchmarks/raw/`.

C. Expected Trends

Based on theoretical predictions from Harchol-Balter et al. [3] and Schroeder and Harchol-Balter [4], we anticipate the following patterns in the baseline data:

- Under the **small-file** workload, mean response time will scale nearly linearly with concurrency and remain low, as all requests are uniformly cheap.
- Under the **large-file** workload, even moderate concurrency will cause mean response time to rise sharply, as each active worker ties up a thread for the full I/O duration.
- Under the **heavy-tailed** workload, mean response time will be dominated by the occasional large-file request blocking small requests behind it—the precise regime where SFF is predicted to help most.

Table IV summarizes the hypothesized improvement of SFF over FCFS for Sprint 4 comparison, drawn from [3].

VI. CONCLUSION

This paper has described the design and baseline measurement of a concurrent POSIX web server implementing the G.5 OSTEP specification. Our system correctly serves static files to concurrent clients using a fixed-size thread pool, a producer-consumer bounded buffer, and configurable FCFS or SFF scheduling. Three UML diagrams—the bounded buffer

TABLE IV
HYPOTHESIZED SFF IMPROVEMENT OVER FCFS (HEAVY-TAILED WORKLOAD)

Load level	FCFS mean RT	Expected SFF gain
Low (< 50% capacity)	Low	Minimal (< 10%)
Med (~70%)	Moderate	~1.5–2×
High (~80%)	High	~3–4×
Near sat. (> 90%)	Very high	> 6×

activity diagram (Figure 1), the HTTP request-response sequence (Figure 2), and the request-to-job lifecycle (Figure 3)—document the critical paths through the system.

The related work survey established that size-based scheduling is theoretically well-motivated (Harchol-Balter [3]; Schroeder [4]), architecturally principled (Welsh [2]; Lampson [8]), and validated at production scale in modern systems (ghOST [9]; Perséphone [10]; Shenango [11]).

In Sprint 4, we will implement SFF scheduling alongside the FCFS baseline, execute the full benchmarking protocol defined in Section IV, populate Table III with measured data, and provide an evidence-based answer to our research question regarding the practical impact of scheduling policy under heavy-tailed web workloads.

REFERENCES

- [1] R. H. Arpaci-Dusseau and A. C. Arpaci-Dusseau, *Operating Systems: Three Easy Pieces*, v1.10. Arpaci-Dusseau Books, 2023. <https://pages.cs.wisc.edu/~remzi/OSTEP/>
- [2] M. Welsh, D. Culler, and E. Brewer, “SEDA: An Architecture for Well-Conditioned, Scalable Internet Services,” in *Proc. 18th ACM Symp. Operating Systems Principles (SOSP ’01)*, Banff, Canada, Oct. 2001, pp. 230–243.
- [3] M. Harchol-Balter, B. Schroeder, N. Bansal, and M. Agrawal, “Size-Based Scheduling to Improve Web Performance,” *ACM Trans. Comput. Syst.*, vol. 21, no. 2, pp. 207–233, May 2003.
- [4] B. Schroeder and M. Harchol-Balter, “Web Servers Under Overload: How Scheduling Can Help,” *ACM Trans. Internet Technol.*, vol. 6, no. 1, pp. 20–52, Feb. 2006.
- [5] M. E. Crovella and A. Bestavros, “Self-Similarity in World Wide Web Traffic: Evidence and Possible Causes,” *IEEE/ACM Trans. Networking*, vol. 5, no. 6, pp. 835–846, Dec. 1997.
- [6] T. E. Anderson, B. N. Bershad, E. D. Lazowska, and H. M. Levy, “Scheduler Activations: Effective Kernel Support for the User-Level Management of Parallelism,” *ACM Trans. Comput. Syst.*, vol. 10, no. 1, pp. 53–79, Feb. 1992.
- [7] R. von Behren, J. Condit, F. Zhou, G. C. Necula, and E. Brewer, “Capriccio: Scalable Threads for Internet Services,” in *Proc. 19th ACM Symp. Operating Systems Principles (SOSP ’03)*, Bolton Landing, NY, Oct. 2003, pp. 268–281.
- [8] B. W. Lampson, “Hints for Computer System Design,” *ACM SIGOPS Oper. Syst. Rev.*, vol. 17, no. 5, pp. 33–48, Oct. 1983.
- [9] J. T. Humphries et al., “ghOST: Fast & Flexible User-Space Delegation of Linux Scheduling,” in *Proc. 28th ACM Symp. Operating Systems Principles (SOSP ’21)*, Virtual, Oct. 2021, pp. 588–604.
- [10] H. M. Demoulin et al., “When Idling Is Ideal: Optimizing Tail-Latency for Heavy-Tailed Datacenter Workloads with Perséphone,” in *Proc. 28th ACM Symp. Operating Systems Principles (SOSP ’21)*, Virtual, Oct. 2021, pp. 621–637.
- [11] A. Ousterhout, J. Fried, J. Behrens, A. Belay, and H. Balakrishnan, “Shenango: Achieving High CPU Efficiency for Latency-sensitive Datacenter Workloads,” in *Proc. 16th USENIX Symp. Networked Systems Design and Implementation (NSDI ’19)*, Boston, MA, Feb. 2019, pp. 361–378.
- [12] W. Glozer, “wrk — a HTTP benchmarking tool,” GitHub, 2023. <https://github.com/wg/wrk>